TL04 Regresión lineal

Índice

1. Introducción
2. Regresión lineal
3. Aplicación a auto-mpg
4. Ejercicio

1 Introducción

Regresión lineal: normal condicional para regresión, $y \in \mathbb{R}$,

$$p(y \mid \boldsymbol{x}, \boldsymbol{\theta}) = \mathcal{N}(y \mid \mu, \sigma^2) \qquad \text{con} \qquad \mu = w_0 + \boldsymbol{w}^t \boldsymbol{x}$$

Ingeniería de características: la entrada $\boldsymbol{x}$ se sustituye por alguna transformación $\boldsymbol{\phi}(\boldsymbol{x})$

Objetivo: aplicar regresión lineal sin y con ingeniería de características

2 Regresión lineal

Regresión lineal: normal condicional para regresión, $y \in \mathbb{R}$,

$$p(y \mid \boldsymbol{x}, \boldsymbol{\theta}) = \mathcal{N}(y \mid \mu, \sigma^2) \qquad \text{con} \qquad \mu = w_0 + \boldsymbol{w}^t \boldsymbol{x}$$

Regresión lineal en scikit-learn: `LinearRegression`

- `fit_interceptbool=True`: añade w_0
- `copy_Xbool=True`: copia el dataset de entrenamiento

Ejemplo:

In [ ]:
```
import numpy as np
from sklearn.linear_model import LinearRegression
X = np.array([[1, 1], [1, 2], [2, 2], [2, 3]])
# y = 1 * x_0 + 2 * x_1 + 3
y = np.dot(X, np.array([1, 2])) + 3
reg = LinearRegression().fit(X, y)
print(reg.score(X, y))
print(reg.coef_)
print(reg.intercept_)
print(reg.predict(np.array([[3, 5]])))
```

```
1.0
[1. 2.]
3.0000000000000018
[16.]
```

3 Aplicación a auto-mpg

auto-mpg: predicción de consumo en miles per gallon

In [ ]:
```python
import numpy as np; import datasets; ds = datasets.load_dataset("scikit-learn/auto-mpg"); ds
```

Out[ ]:
```
DatasetDict({
    train: Dataset({
        features: ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight', 'acceleration', 'model year', 'origi
n', 'car name'],
        num_rows: 398
    })
})
```

Formato: conversión a `dataframe` de `pandas`

In [ ]:
```python
import pandas as pd; df = ds['train'].to_pandas(); df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   mpg           398 non-null    float64
 1   cylinders     398 non-null    int64
 2   displacement  398 non-null    float64
 3   horsepower    398 non-null    object
 4   weight        398 non-null    int64
 5   acceleration  398 non-null    float64
 6   model year    398 non-null    int64
 7   origin        398 non-null    int64
 8   car name      398 non-null    object
dtypes: float64(3), int64(4), object(2)
memory usage: 28.1+ KB
```

Preproceso: tras examinar variables en el Dataset Viewer

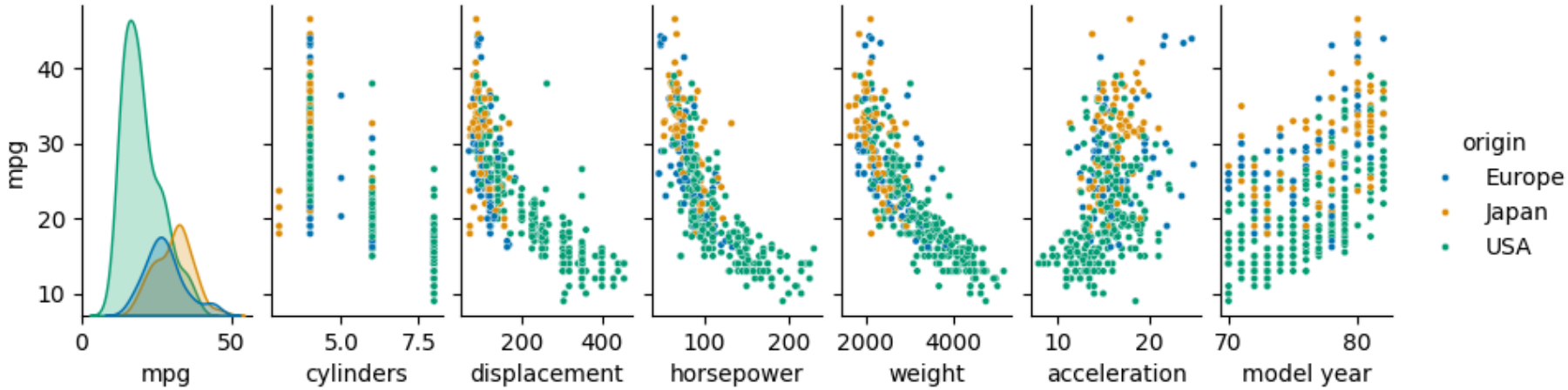
- `horsepower`: normalización de tipo tras eliminar seis casos con `"?"`
- `origin`: normalización de tipo ya que es una categórica con tres posibles valores; 1, 2 o 3
- `car name`: eliminación ya que el modelo de coche no interesa como variable explicativa

In [ ]:
```
df.drop(df[df.horsepower == "?"].index, inplace=True)
df = df.astype({'horsepower' : np.int64})
df['origin'] = df['origin'].map({1: 'USA', 2: 'Europe', 3: 'Japan'})
df = df.astype({'origin' : 'category'})
df.drop('car name', axis=1, inplace=True)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 392 entries, 0 to 397
Data columns (total 8 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   mpg           392 non-null    float64
 1   cylinders     392 non-null    int64
 2   displacement  392 non-null    float64
 3   horsepower    392 non-null    int64
 4   weight        392 non-null    int64
 5   acceleration  392 non-null    float64
 6   model year    392 non-null    int64
 7   origin        392 non-null    category
dtypes: category(1), float64(3), int64(4)
memory usage: 25.0 KB
```

Inspección visual:

In [ ]:
```
import seaborn as sns
sns.pairplot(df, y_vars='mpg', hue="origin", aspect=.5, palette='colorblind', plot_kws=dict(marker="o", s=10));
```



Diseño experimental:

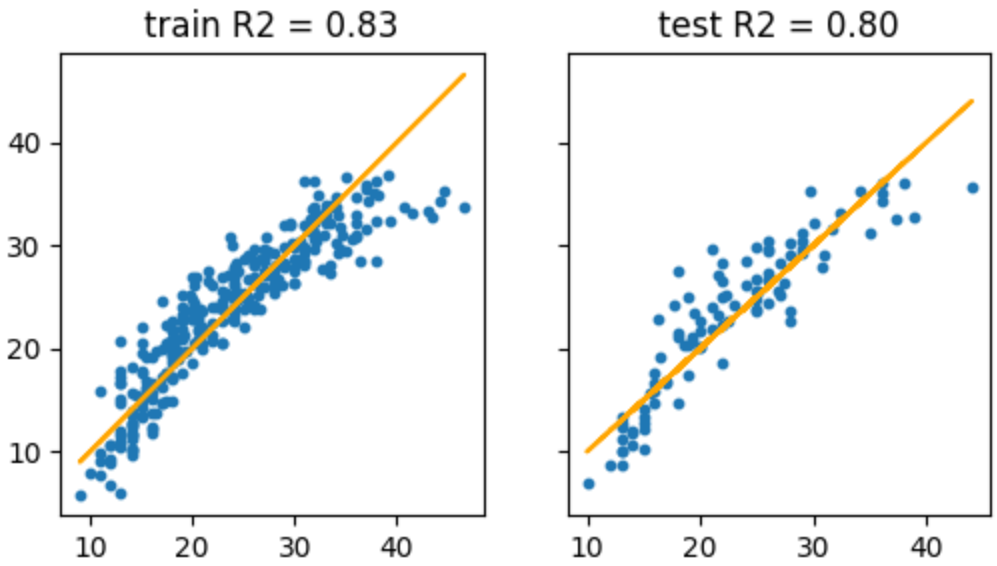
- Binarización de `origin` para facilitar su tratamiento
- Partición de los datos en entrenamiento y test
- Estandarización de características

In [ ]:
```
from sklearn.preprocessing import StandardScaler; from sklearn.model_selection import train_test_split
df = pd.concat([df, pd.get_dummies(df.origin, prefix='', prefix_sep='')], axis=1)
df.drop('origin', axis=1, inplace=True); y = df.pop('mpg'); X = df
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
scaler = StandardScaler(); X_train = scaler.fit_transform(X_train); X_test = scaler.transform(X_test);
```

Regresión lineal sin ingeniería de características:

In [ ]:
```
from sklearn.linear_model import LinearRegression; from sklearn.metrics import r2_score
linreg = LinearRegression().fit(X_train, y_train)
y_train_pred = linreg.predict(X_train); y_test_pred = linreg.predict(X_test)
r2_train = r2_score(y_train, y_train_pred); r2_test = r2_score(y_test, y_test_pred)
```

In [ ]:
```
import matplotlib.pyplot as plt
fig, axs = plt.subplots(1, 2, figsize=(6, 3), sharey = 'all')
ax = axs[0]; ax.set_title(f'train R2 = {r2_train:.2f}')
ax.scatter(y_train, y_train_pred, s=10); ax.plot(y_train, y_train, color='orange')
ax = axs[1]; ax.set_title(f'test R2 = {r2_test:.2f}')
ax.scatter(y_test, y_test_pred, s=10); ax.plot(y_test, y_test, color='orange');
```

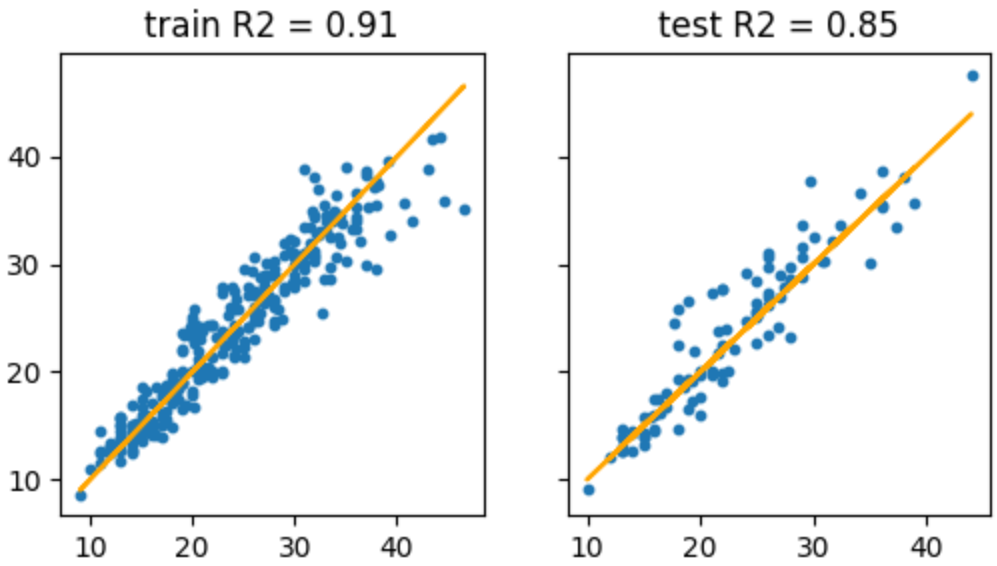


R2 mide que tan buena es la aproximación.
Max R2 = 1
Puede ser negativo

Regresión lineal con ingeniería de características:

In [ ]:
```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(2); X_train = poly.fit_transform(X_train); X_test = poly.transform(X_test)
linreg = LinearRegression().fit(X_train, y_train)
y_train_pred = linreg.predict(X_train); y_test_pred = linreg.predict(X_test)
r2_train = r2_score(y_train, y_train_pred); r2_test = r2_score(y_test, y_test_pred)
```

In [ ]:
```
fig, axs = plt.subplots(1, 2, figsize=(6, 3), sharey = 'all')
ax = axs[0]; ax.set_title(f'train R2 = {r2_train:.2f}')
ax.scatter(y_train, y_train_pred, s=10); ax.plot(y_train, y_train, color='orange')
ax = axs[1]; ax.set_title(f'test R2 = {r2_test:.2f}')
ax.scatter(y_test, y_test_pred, s=10); ax.plot(y_test, y_test, color='orange');
```



Con ingeniería de características, conseguimos mejorar la aproximación.

4 Ejercicio

Ejercicio: aplica regresión lineal sin y con ingeniería de características a los datasets siguientes

- scikit-learn/Fish para predecir el peso de un pez
- scikit-learn/tips para predecir la propina de un cliente de restaurante

Aplicación a Fish: R2 en test: sin 0.95, con 0.96

Aplicación a tips: R2 en test: sin 0.55, con 0.51

Aplicación a Fish

In [ ]:
```
import numpy as np; import datasets; import pandas as pd;
df = datasets.load_dataset("scikit-learn/Fish")['train'].to_pandas(); df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 159 entries, 0 to 158
Data columns (total 7 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   Species  159 non-null    object
 1   Weight   159 non-null    float64
 2   Length1  159 non-null    float64
 3   Length2  159 non-null    float64
 4   Length3  159 non-null    float64
 5   Height   159 non-null    float64
 6   Width    159 non-null    float64
dtypes: float64(6), object(1)
memory usage: 8.8+ KB
```

In [ ]:
```
from sklearn.preprocessing import StandardScaler; from sklearn.model_selection import train_test_split
df = df.astype({'Species' : 'category'})
df = pd.concat([df, pd.get_dummies(df.Species, prefix='', prefix_sep='')], axis=1)
df.drop('Species', axis=1, inplace=True); y = df.pop('Weight'); X = df
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
scaler = StandardScaler(); X_train = scaler.fit_transform(X_train); X_test = scaler.transform(X_test);
```

In [ ]:
```
from sklearn.linear_model import LinearRegression; from sklearn.metrics import r2_score
linreg = LinearRegression().fit(X_train, y_train)
y_train_pred = linreg.predict(X_train); y_test_pred = linreg.predict(X_test)
sin_r2_train = r2_score(y_train, y_train_pred); sin_r2_test = r2_score(y_test, y_test_pred)
```

In [ ]:
```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(2); X_train = poly.fit_transform(X_train); X_test = poly.transform(X_test)
linreg = LinearRegression().fit(X_train, y_train)
y_train_pred = linreg.predict(X_train); y_test_pred = linreg.predict(X_test)
con_r2_train = r2_score(y_train, y_train_pred); con_r2_test = r2_score(y_test, y_test_pred)
print(f'R2: sin {sin_r2_test:.2f} con {con_r2_test:.2f} (train: sin {sin_r2_train:.2f} con {con_r2_train:.2f})')
```

```
R2: sin 0.95 con 0.96 (train: sin 0.93 con 0.99)
```

Aplicación a tips

In [ ]:
```
import numpy as np; import datasets; import pandas as pd;
df = datasets.load_dataset("scikit-learn/tips")['train'].to_pandas(); df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 244 entries, 0 to 243
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   total_bill  244 non-null    float64
 1   tip         244 non-null    float64
 2   sex         244 non-null    object
 3   smoker      244 non-null    object
 4   day         244 non-null    object
 5   time        244 non-null    object
 6   size        244 non-null    int64
dtypes: float64(2), int64(1), object(4)
memory usage: 13.5+ KB
```

In [ ]:
```
df.sex = df.sex.map({'Male': 0,'Female': 1}).astype('bool')
df.smoker = df.smoker.map({'No': 0,'Yes': 1}).astype('bool')
df = df.astype({'day':'category'}); df = pd.concat([df, pd.get_dummies(df.day, prefix='', prefix_sep='')], axis=1)
df.drop('day', axis=1, inplace=True); df.time = df.time.map({'Dinner': 0, 'Lunch': 1}).astype('bool')
y = df.pop('total_bill'); X = df
```

In [ ]:
```
from sklearn.model_selection import train_test_split; from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
linreg = LinearRegression().fit(X_train, y_train)
y_train_pred = linreg.predict(X_train); y_test_pred = linreg.predict(X_test)
sin_r2_train = r2_score(y_train, y_train_pred); sin_r2_test = r2_score(y_test, y_test_pred)
X_train = np.c_[(np.ones((X_train.shape[0], 1)), X_train, np.square(X_train))]
X_test = np.c_[(np.ones((X_test.shape[0], 1)), X_test, np.square(X_test))]
linreg = LinearRegression().fit(X_train, y_train)
y_train_pred = linreg.predict(X_train); y_test_pred = linreg.predict(X_test)
con_r2_train = r2_score(y_train, y_train_pred); con_r2_test = r2_score(y_test, y_test_pred)
print(f'R2: sin {sin_r2_test:.2f} con {con_r2_test:.2f} (train: sin {sin_r2_train:.2f} con {con_r2_train:.2f})')
```

```
R2: sin 0.55 con 0.51 (train: sin 0.58 con 0.59)
```